

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



DEPARTMENT OF COMPUTER NETWORKS AND TELECOMMUNICATION

REPORT OF PROJECT

Project: Video Streaming System using RTSP/RTP protocols

Lecture:	Lê Hà Minh	
Leader:	Khang Phuc Nguyen	24120068
Members:	Nghia Trong Hoang	24120103
	Trong Minh Pham	24120151

Ho Chi Minh City, 12/2025

Table of Contents

1	Cơ sở lý thuyết	2
1.1	Socket và mô hình tiến trình giao tiếp	2
1.2	Tầng giao vận: TCP và UDP	2
1.3	RTSP (Real-Time Streaming Protocol)	3
1.4	RTP (Real-Time Transport Protocol)	3
1.5	RTCP (RTP Control Protocol) - tóm tắt	4
1.6	Luồng hoạt động tổng quát	4
1.7	Bộ đệm, tái lắp ráp và chống jitter	4
1.8	Phân mảnh và MTU	4
1.9	Threading và timeout	5
1.10	NAT và triển khai thực tế	5
1.11	Thiết kế hướng đối tượng	5
1.12	Tóm lược	6
1.13	Tổng hợp lý thuyết qua bảng tóm tắt	6



Giới thiệu

Với sự phát triển của công nghệ số và nhu cầu giải trí của con người tăng cao, đặc biệt khi cuộc cách mạng 4.0, sự bùng nổ của Internet thì vai trò của Mạng Máy Tính trở nên ngày càng quan trọng và có nhiều ứng dụng thực tiễn chẳng hạn là như là nhắn tin trực tuyến, làm việc trực tuyến và đặc biệt là trao đổi các thông tin, dữ liệu thông quan mạng. Đặc biệt trong số đó thì việc truyền các Video hoặc Ảnh thông qua được sử dụng ngày phổ biến trong các ứng dụng mạng xã hội như Facebook, TikTok và Youtube.

Trong dự án này, nhóm đã tìm hiểu và thực hiện hóa một hệ thống đơn giản cho phép hai ứng dụng mạng có thể giao tiếp với nhau thông qua giao tiếp RTSP/RTP để có thể phát một video trực tuyến. Dự án này được làm với mục đích học tập và hiểu rõ hơn về các giao thức cũng như là cách hoạt động của các ứng dụng trong thực tế, mặt khác, dự án này cũng chỉ là mô phỏng lại cách phát trực tiếp một video giữa các ứng dụng nên được xây dựng không quá phức tạp và tối ưu.

Toàn bộ mã nguồn của dự án được xây dựng bằng ngôn ngữ `C++ 17`, sử dụng `CMake` để có thể build trên cả Window và Linux, cũng như là sử dụng các thư viện `winsock2` và `POSIX sockets` để tạo các socket cho việc giao tiếp giữa các ứng dụng. Bên cạnh đó, nhóm cũng đã áp dụng các kiến thức về `Design Pattern` cho dự án với mục đích là mã nguồn của dự án có thể tái sử dụng, sạch sẽ và dễ đọc, dễ sửa lỗi cũng như là tối ưu trong quá trình hoạt động.

Để có thể có một giao diện cho ứng dụng của bên phía `client` thì nhóm đã sử dụng `Qt6` để tạo ra một giao diện đơn giản và dễ sử dụng cho người dùng. Toàn bộ quá trình, giao thức, phản hồi và yêu cầu của `server` và `client` được dựa trên toàn bộ yêu cầu được đính kèm trong file `Socket_2526` của thầy Lê Hà Minh [1]. Nhóm xin chân thành cảm ơn thầy đã hỗ trợ và cung cấp tài liệu để nhóm có thể hoàn thành đồ án này một cách tốt đẹp nhất.

Nhóm Trưởng

(Chữ ký và họ tên)

Khang

Nguyễn Phúc Khang

Ho Chi Minh City, 12/2025

1 Cơ sở lý thuyết

Dự án hiện thực một hệ thống truyền phát video thời gian thực (RTSP/RTP) với các thành phần:

Thành phần	Mô tả
Kênh điều khiển	RTSP/TCP để thiết lập, điều khiển phiên (SETUP, PLAY, PAUSE, TEARDOWN)
Kênh dữ liệu	RTP/UDP để vận chuyển khung hình, hỗ trợ phân mảnh, đánh số thứ tự, đánh dấu thời gian
Bộ đệm phía client	Tiền tải (prebuffer), phát mượt, phục hồi gói đến không đúng thứ tự
Thiết kế hướng đối tượng	Đóng gói Socket, RTSP client/server, RTP encode/decode, chiến lược mã hoá/fragmentation

1.1 Socket và mô hình tiến trình giao tiếp

Mọi trao đổi giữa hai tiến trình mạng đều đi qua **Socket** [2]. Socket là API cung cấp các chức năng sau:

Chức năng	Mô tả
Khởi tạo và liên kết	Tạo, bind, listen/accept (TCP) hoặc bind (UDP)
Truyền nhận dữ liệu	Gửi/nhận byte hoặc datagram; thiết lập timeout; SO_REUSEADDR để tái sử dụng cổng
Giải phóng tài nguyên	Đóng kết nối, giải phóng tài nguyên

Trong dự án, lớp **Socket** tự bọc các thao tác này cho cả TCP (RTSP) và UDP (RTP), kèm timeout nhỏ để thread nhận có thể thoát khi dừng.

1.2 Tầng giao vận: TCP và UDP

TCP (Transmission Control Protocol) là giao thức hướng kết nối, tin cậy, có kiểm soát luồng và tắc nghẽn [2], phù hợp cho kênh điều khiển RTSP để bảo đảm lệnh đến đúng và đủ. **UDP** (User Datagram Protocol) là giao thức phi kết nối, không đảm bảo, có độ trễ thấp [2], phù hợp cho kênh dữ liệu RTP: chấp nhận mất mát nhỏ để giữ độ trễ thấp.

1.3 RTSP (Real-Time Streaming Protocol)

Theo RFC 2326 [3], RTSP là “bộ điều khiển từ xa” cho media server. Phiên (session) được định danh bằng Session ID và tồn tại qua nhiều kết nối TCP.

Phương thức	Mô tả
SETUP	Client yêu cầu thiết lập, cung cấp cổng RTP/RTCP (client_port), server phản hồi với Session ID và server_port
PLAY	Bắt đầu phát; server gửi RTP qua UDP tới client_port; có thể kèm Range
PAUSE	Dừng tạm, giữ phiên
TEARDOWN	Kết thúc, giải phóng tài nguyên
Các trường chung	Mọi yêu cầu kèm CSeq (tăng dần) và Session (sau SETUP)

Trong code: `RTSPClient` kết nối TCP “lười” (lazy), gửi chuỗi RTSP, parse mã trạng thái, trích Session, server_port, tổng số frame (X-Total-Frames). `ServerWorker` parse yêu cầu, phản hồi với Session, Transport và tổng frame, spawn thread gửi RTP khi PLAY.

1.4 RTP (Real-Time Transport Protocol)

Theo RFC 3550 [4], RTP gói dữ liệu thời gian thực. Header 12 byte gồm:

Trường	Mô tả
Version, P, X, CC	Version=2, Padding, Extension, CSRC count
M (Marker)	Đánh dấu cuối frame (marker=1 cho gói cuối)
PT (Payload Type)	Loại payload (MJPEG=26)
Sequence Number	16-bit, phát hiện mất/đảo gói
Timestamp	32-bit, đồng bộ và tái tạo tốc độ phát
SSRC	32-bit, định danh nguồn

Trong dự án:

Thành phần	Chi tiết triển khai
Đóng gói RTP	<code>RTPPacket::encode</code> đóng gói header + payload; <code>RTPPacketBuilder</code> hỗ trợ fluent API
Payload MJPEG	Mỗi frame JPEG được gửi trọn hoặc phân mảnh (HDEncodingStrategy) theo MTU 1400B; marker=1 cho mảnh cuối
Quản lý sequence/timestamp	Server tăng <code>sequenceNumber</code> theo số gói đã gửi, <code>timestamp</code> tăng đều để tạo nhịp phát

1.5 RTCP (RTP Control Protocol) - tóm tắt

Dự án không hiện thực RTCP, nhưng lý thuyết RTCP [4] cung cấp thông kê mất gói, jitter, đồng bộ A/V. Nếu cần mở rộng, có thể thêm gói SR/RR (Sender Report/Receiver Report) từ client về server để điều chỉnh tốc độ, báo mất gói.

1.6 Luồng hoạt động tổng quát

Bước	Hoạt động
1	Client khởi tạo RTSP/TCP, gửi SETUP kèm cổng RTP (client_port)
2	Server mở VideoStream, đếm tổng frame, phản hồi 200 OK với Session ID, server_port, X-Total-Frames
3	Client mở UDP socket (bind client_port), khởi tạo receiver và bộ đệm, chuyển state READY
4	Client gửi PLAY ; server chuyển state PLAYING, tạo thread gửi RTP gói-gói theo nhịp Timer (25fps)
5	Client nhận RTP qua UDP, reassemble theo timestamp + marker, đẩy frame vào FrameBuffer; UI đọc buffer và hiển thị
6	PAUSE : server ngừng gửi; PLAY tiếp tục cùng Session
7	TEARDOWN : hai bên giải phóng tài nguyên, socket đóng

1.7 Bộ đệm, tái lắp ráp và chống jitter

Thành phần	Chức năng
FrameBuffer	Hàng đợi giới hạn (mặc định 500 frame); hỗ trợ push/pop blocking với timeout
FrameReassembler	Nhóm các gói cùng timestamp, sort theo sequence, ghép payload, marker=1 kết thúc frame
Prebuffer	Chờ đủ N frame trước khi phát để giảm giật
Thích ứng	Nếu buffer thấp, UI giảm tốc độ đọc (tăng interval), hoặc chờ thêm; nếu hết frame $\Rightarrow$ chuyển READY, giữ Session để PLAY lại

1.8 Phân mảnh và MTU

Mạng thường giới hạn MTU (Maximum Transmission Unit) khoảng 1500 byte [2]. Dự án đặt payload RTP tối đa 1400 byte, phần còn lại dành cho header IP/UDP/RTP. HDEncodingStrategy cắt frame lớn thành nhiều gói, marker=1 cho mảnh cuối, tất cả mảnh dùng chung timestamp để client có thể tái lắp ráp đúng.



1.9 Threading và timeout

Bên	Cấu hình thread và timeout
Server	Một thread RTSP xử lý lệnh; một thread streaming gửi RTP khi state PLAYING
Client	Một thread nhận UDP (<code>RTPReceiver</code>), một thread UI Qt render; đồng bộ qua buffer
Socket timeout	Timeout nhỏ (5ms cho UDP) để vòng lặp nhận kiểm tra cờ dừng; timeout RTSP (TCP) lớn hơn
Reuse address	Bật <code>SO_REUSEADDR</code> để tránh kẹt cổng khi TEARDOWN/PLAY lại

1.10 NAT và triển khai thực tế

Môi trường	Yêu cầu triển khai
RTSP/TCP	Mặc định 554, ở đây dùng 8554; cần mở trên firewall/NAT nếu chạy qua Internet
RTP/UDP	Cổng do client chọn, cần mở trên firewall/NAT nếu chạy qua Internet
Trong LAN	Không cần port forwarding; dùng trực tiếp IP nội bộ
Qua Internet	Cần NAT mapping hoặc giải pháp VPN/tunnel (Tailscale, v.v.) nếu không mở port

1.11 Thiết kế hướng đối tượng

Lớp/Module	Vai trò
Socket	Đóng gói TCP/UDP, timeout, send/recv
RTSPClient/ServerWorker	Quản lý phiên, state machine RTSP
RTPPacket, RTPPacketBuilder	Xây gói RTP
EncodingStrategy (SD/HD)	Pattern Strategy để chọn cách phân mảnh
FrameBuffer, FrameReassembler	Tách trách nhiệm nhận và tái lắp khung
UI Qt	Xử lý sự kiện SETUP/PLAY/PAUSE/TEARDOWN, hiển thị trạng thái, buffer

1.12 Tóm lược

Dự án kết hợp kênh điều khiển tin cậy (RTSP/TCP [3]) với kênh dữ liệu thời gian thực (RTP/UDP [4]), bổ sung cơ chế phân mảnh, đánh dấu thời gian, đánh số gói, bộ đệm và tái lắp khung để đảm bảo phát video mượt, giảm jitter. Các quyết định kỹ thuật (timeout ngắn cho UDP, prebuffer, strategy fragmentation) bám sát đặc thù ứng dụng streaming thời gian thực [2].

1.13 Tổng hợp lý thuyết qua bảng tóm tắt

Các bảng dưới đây gom kiến thức chính theo dạng so sánh, tránh liệt kê dài.

Tiêu chí	TCP	UDP	Lý do chọn trong dự án
Kết nối	Hướng kết nối, bắt tay 3 bước	Phi kết nối	RTSP cần phiên tin cậy; RTP cần độ trễ thấp
Tin cậy	Có, retransmission	Không	Chấp nhận mất gói nhỏ để giữ latency
Kiểm soát tắc nghẽn	Có	Không	Điều khiển dùng TCP; dữ liệu ưu tiên tốc độ
Độ trễ	Cao hơn	Thấp	UDP phù hợp streaming
Kiểm soát luồng	Có	Không	Không cần ở kênh dữ liệu

1.13.0.1 So sánh TCP và UDP trong truyền thông thời gian thực

Phương thức	Mục đích	Trường trọng yếu
SETUP	Thiết lập phiên, thương lượng cổng	CSeq, Transport (client_port), phản hồi Session, server_port, tổng frame
PLAY	Bắt đầu phát	CSeq, Session, (tùy chọn Range)
PAUSE	Tạm dừng, giữ phiên	CSeq, Session
TEARDOWN	Kết thúc, giải phóng	CSeq, Session

1.13.0.2 Các phương thức RTSP chủ đạo

Trường	Kích thước	Chức năng
Version	2 bit	Luôn bằng 2
P, X, CC	1,1,4 bit	Padding, Extension, CSRC count
Marker (M)	1 bit	Đánh dấu gói cuối frame
Payload Type	7 bit	MJPEG = 26
Sequence Number	16 bit	Phát hiện mất/đảo gói
Timestamp	32 bit	Đồng bộ tốc độ phát
SSRC	32 bit	Định danh nguồn gửi

1.13.0.3 Header RTP (12 byte chuẩn) Trong triển khai, payload giới hạn 1400 byte để tránh vượt MTU [2]; tất cả mảnh chung timestamp, gói cuối đặt Marker=1. Payload type 26 dành cho MJPEG tuân theo RTP payload format [5].

Thành phần	Thiết lập	Lý do
UDP socket (RTP)	Timeout 5 ms	Vòng lặp nhận thoát nhanh khi dừng
TCP socket (RTSP)	Timeout dài hơn	Giữ kết nối giữa các lệnh
SO_REUSEADDR	Bật	Giảm kẹt cổng khi PLAY/TEAR-DOWN liên tục
Bind client_port	Bắt buộc	Nhận gói RTP từ server

1.13.0.4 Socket và thiết lập thời gian chờ

1.13.0.5 Bộ đệm và tái lắp ráp



Thành phần	Vai trò	Ghi chú
FrameBuffer	Hàng đợi khung, chặn khi đầy/rỗng	Giảm jitter, pre-buffer trước khi phát
FrameReassembler	Gom gói cùng timestamp, sắp xếp theo seq	Marker=1 kết thúc frame
Prebuffer	Chờ đủ N khung rồi phát	Tránh giật hình khi mạng dao động



List of Figures



List of Tables

References

- [1] L. H. Minh, “Socket programming assignment: Rtp video streaming,” Course Material, 2024, document Socket_2526, Ho Chi Minh University of Science.
- [2] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Boston, MA: Pearson, 2021, global Edition.
- [3] H. Schulzrinne, A. Rao, and R. Lanphier, “Real Time Streaming Protocol (RTSP),” RFC 2326, Apr. 1998. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc2326>
- [4] H. Schulzrinne, S. L. Casner, R. Frederick, and V. Jacobson, “RTP: A Transport Protocol for Real-Time Applications,” IETF, RFC 3550, Jul. 2003, truy cập ngày 13/12/2025. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc3550>
- [5] L. M. Berc, W. C. Fenner, R. Frederick, and S. McCanne, “RTP Payload Format for JPEG-compressed Video,” IETF, RFC 2435, Oct. 1998. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc2435>